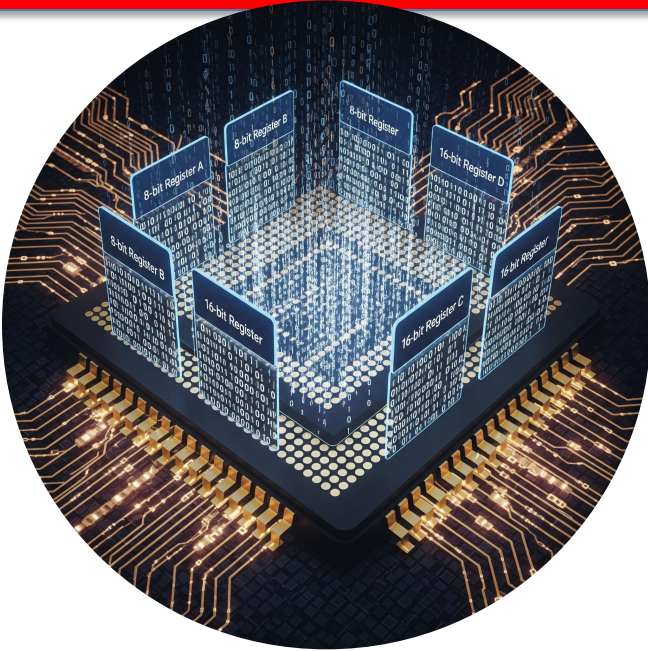




```
edit: C:\emu8086\examples\1_sample.asm
file edit bookmarks assembler emulator math ascii codes help
new open examples save compile emulate calculator convertor options help about

org 100h
; set video mode
mov ax, 3          ; text mode 80x25, 16 colors, 8 pages (ah=0, al=3)
int 10h           ; do it!

; cancel blinking and enable all 16 colors:
mov ax, 1003h
mov bx, 0
int 10h

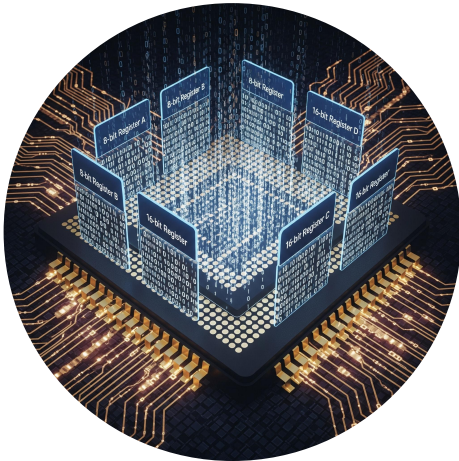
; set segment register:
mov ax, 0b800h
mov ds, ax

; print "hello world"
; first byte is ascii code, second byte is color code.
mov [02h], 'H'
mov [04h], 'e'
mov [06h], 'l'
mov [08h], 'l'
mov [0ah], 'o'
```

INTRODUCCIÓN A LA PROGRAMACIÓN DE ENSAMBLADOR 8088/8086

Resumen de Registros

- La CPU opera en binario a alta velocidad
- El lenguaje ensamblador usa mnemónicos para código máquina
- Los registros 8088/86 pueden ser de 8 o 16 bits
- La pila es memoria temporal para la CPU



8086 Microprocessor Emulator

File Math Debug View Virtual Devices Virtual Drive Help

Load Reload Single Step Run step delay ms: 200

Registers

	H	L
AX	00	00
BX	00	00
CX	00	00
DX	00	00
CS	0050	
IP	0000	
SS	0000	
SP	0000	
BP	0000	
SI	0000	
DI	0000	
DS	0000	
ES	0000	

memory (1K) at: 0050 : 0000

Disassemble: 0050 : 0000

Offset	Hex	Dec	ASCII
0000:	00	000	
0001:	00	000	
0002:	00	000	
0003:	00	000	
0004:	00	000	
0005:	00	000	
0006:	00	000	
0007:	00	000	
0008:	00	000	
0009:	00	000	
000A:	00	000	
000B:	00	000	
000C:	00	000	
000D:	00	000	

ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL

User Screen Actual Source ALU Stack FLAGS

INTRODUCCIÓN A LA PROGRAMACIÓN DE ENSAMBLADOR

La CPU solo puede funcionar en binario a velocidades muy altas.

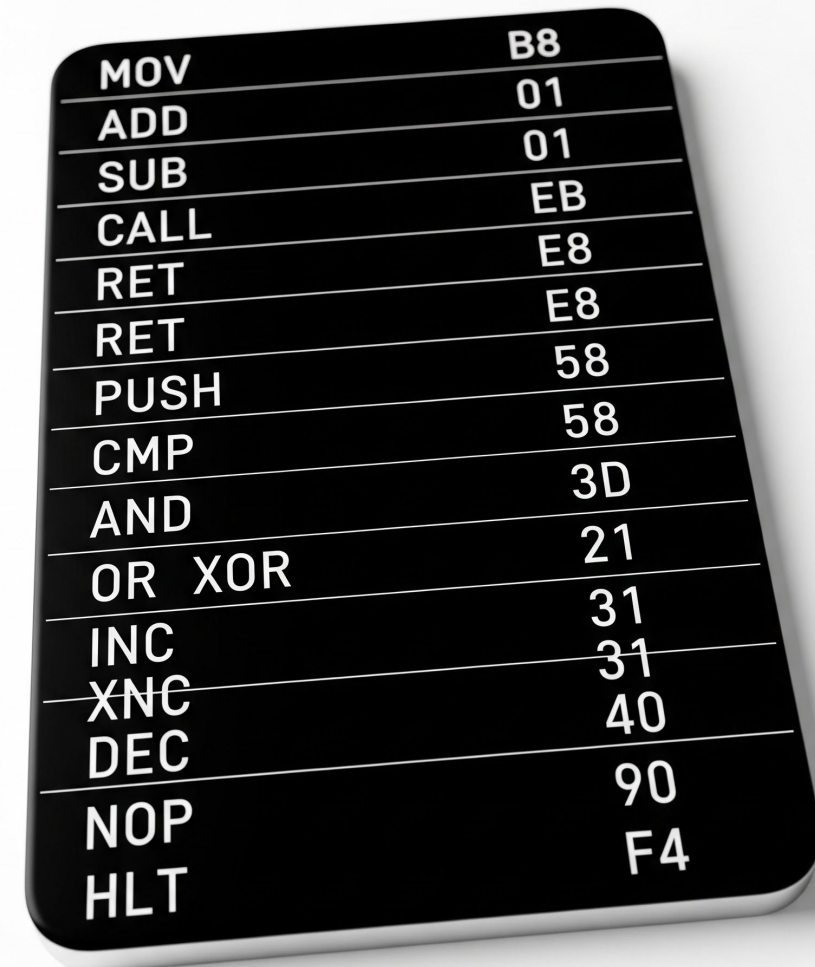
– Es tedioso y lento para los humanos lidiar **con 0s y 1s** para programar la computadora

Un programa de 0s & 1s se llama **lenguaje de máquina**.

Los primeros programadores de computadoras codificaron programas en lenguaje de máquina

Eventualmente se desarrollaron **lenguajes ensambladores**, que proporcionaron **mnemotecnica** para el código máquina.

La mnemotécnica se usa típicamente en la literatura de informática e ingeniería para referirse a códigos y abreviaturas que son relativamente fáciles de recordar.



MOV	B8
ADD	01
SUB	01
CALL	EB
RET	E8
RET	E8
PUSH	58
CMP	58
AND	3D
OR XOR	21
INC	31
XNC	31
DEC	40
NOP	90
HLT	F4

INTRODUCCIÓN A LA PROGRAMACIÓN DE ENSAMBLADOR

El lenguaje ensamblador se conoce como un lenguaje de bajo nivel porque trata directamente con la **estructura interna de la CPU**.

Los programas en lenguaje ensamblador deben ser traducidos a código máquina por un programa llamado **ensamblador**.

Para programar en lenguaje ensamblador, los programadores deben conocer **el número de registros y su tamaño**, así como otros detalles de la CPU.

Hoy existen diferentes lenguajes de programación como C, C++, Java, etc. Se llaman lenguajes de alto nivel porque el programador no tiene que preocuparse **por los detalles internos** de la CPU.

Los lenguajes de alto nivel son traducidos a código máquina por un programa llamado **compilador**.

Para escribir un programa en C, uno debe usar un **compilador de C** para traducir el programa al lenguaje de máquina

INTRODUCCIÓN A LA PROGRAMACIÓN DE ENSAMBLADOR

Un programa en lenguaje ensamblador consiste en una serie de líneas de instrucciones en lenguaje ensamblador.

- Una instrucción en lenguaje ensamblador consiste en una **mnemotécnica**, opcionalmente seguida de uno o dos Operandos.
 - Los operandos son los elementos de datos que se manipulan.
 - Los mnemotécnicos son comandos a la CPU, diciéndole qué hacer con esos operandos.
- Dos instrucciones ampliamente utilizadas son **mover & sumar**.

La instrucción **MOV** copia datos de una ubicación a otra, utilizando este formato:

```
MOV destination,source ;copy source operand to destination
```

Esta instrucción le dice a la CPU que mueva (en realidad, copie) el operando de origen al operando de destino.

- Por ejemplo, la instrucción "MOV DX,CX" copia el contenido del registro CX al registro DX

Instrucción MOV

- Este programa primero carga CL con el valor 55H, luego mueve este valor a varios registros dentro de la CPU.

```
MOV CL,55H ;move 55H into register CL
MOV DL,CL ;copy the contents of CL into DL (now DL=CL=55H)
MOV AH,DL ;copy the contents of DL into AH (now AH=DL=55H)
MOV AL,AH ;copy the contents of AH into AL (now AL=AH=55H)
MOV BH,CL ;copy the contents of CL into BH (now BH=CL=55H)
MOV CH,BH ;copy the contents of BH into CH (now CH=BH=55H)
```

- Los valores **NO** se pueden cargar directamente en ningún registro de segmento (CS, DS, ES o SS).
 - Para cargar un valor en un registro de segmento, cárguelo en un **registro que no sea de segmento** y, a continuación, muévelo al registro de segmento.

```
MOV AX,2345H ;load 2345H into AX
MOV DS,AX ;then load the value of AX into DS

MOV DI,1400H ;load 1400H into DI
MOV ES,DI ;then move it into ES, now ES=DI=1400
```

Instrucción ADD

- Tiene el siguiente formato:
- **ADD destino, fuente** : Suma el Operando fuente al destino
- Para agregar dos números, como 25H y 34H, cada uno se puede mover a un registro y luego sumar:

```
MOV    AL,25H    ;move 25 into AL
MOV    BL,34H    ;move 34 into BL
ADD    AL,BL     ;AL = AL + BL
```

Instrucción ADD

- No es necesario mover los datos a los registros antes de sumarlos:

```
MOV DH,25H    ;load one operand into DH
ADD DH,34H    ;add the second operand to DH
```

- Un registro de 8 bits puede contener números hasta FFH.

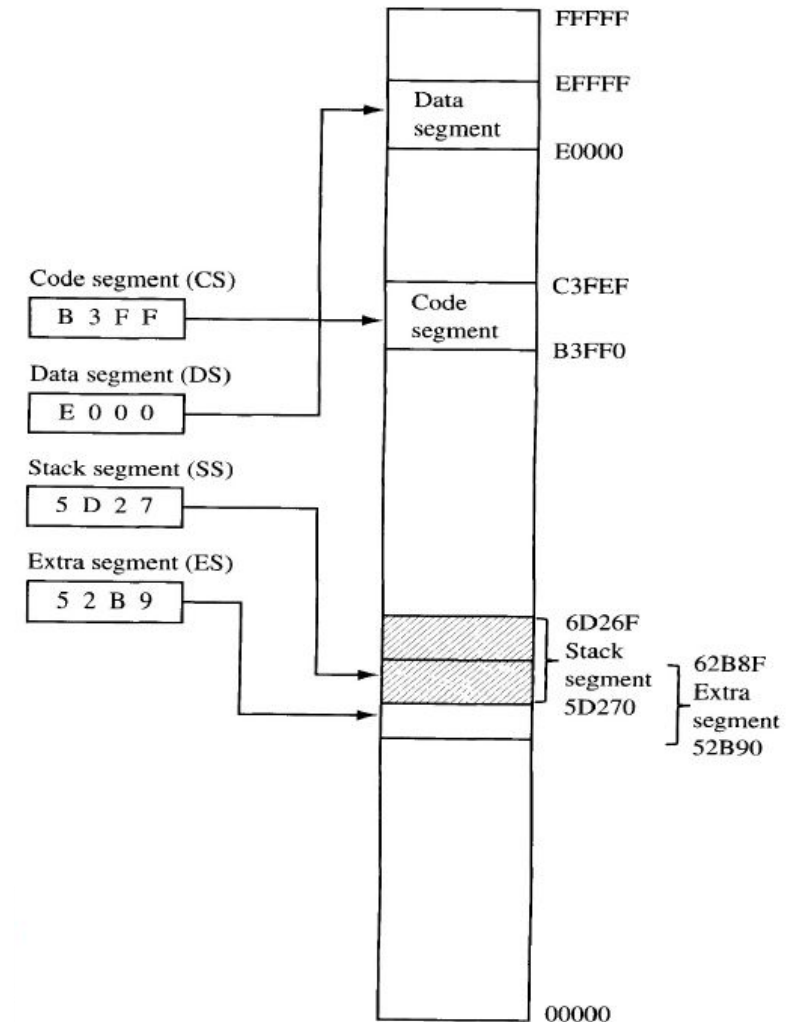
– Para números mayores que FFH (255 decimales), se debe usar un registro de 16 bits como AX, BX, CX o DX.

- El siguiente


```
MOV AX,34EH    ;move 34EH into AX
MOV DX,6A5H    ;move 6A5H into DX
ADD DX,AX      ;add AX to DX: DX = DX + AX
```

INTRODUCCIÓN A LOS SEGMENTOS DEL PROGRAMA

- Un típico programa de lenguaje ensamblador consta de al menos tres segmentos:
 - **Un segmento de código** - que contiene las instrucciones en lenguaje ensamblador que realizan las tareas para las que el programa fue diseñado.
 - **Un segmento de datos** - utilizado para almacenar información (datos) para ser procesada por las instrucciones en el segmento de código.
 - **Un segmento de pila** - utilizado por la CPU para almacenar información temporalmente.



Dirección lógica y dirección física

- En la literatura sobre 8086, hay tres tipos de direcciones mencionadas con frecuencia:
 - **La dirección física** - la dirección de **20 bits** en los pines de dirección del procesador 8086, *decodificada* por los circuitos de interfaz de memoria.
Esta dirección puede tener un rango de 00000H a FFFFFFFH.
Una ubicación física real en RAM o ROM dentro del rango de memoria de 1 mb.
 - **La dirección de desplazamiento** - una ubicación en un rango de segmento de 64K bytes, que puede variar de 0000H a FFFFH.
 - **La dirección lógica** - que consiste en un valor de segmento y una dirección de desplazamiento.

Dirección lógica y dirección física

- Las direcciones dentro de un segmento pueden variar de la dirección 0 a la dirección FFFFh. Esto corresponde a la longitud de 64Kbyte del segmento llamado desplazamiento
Una dirección dentro de un segmento se denomina **dirección lógica**
Ej. La dirección lógica 0005h en el segmento de código en realidad corresponde a $B3FF0h + 5 = B3FF5h$.

- Ejemplo 1:**
Valor base del segmento: 1234h
Desplazamiento: 0022h

Dir Física es $12340h + 0022h = 12362h$

Dir lógica: 1234:0022

Segmento de código

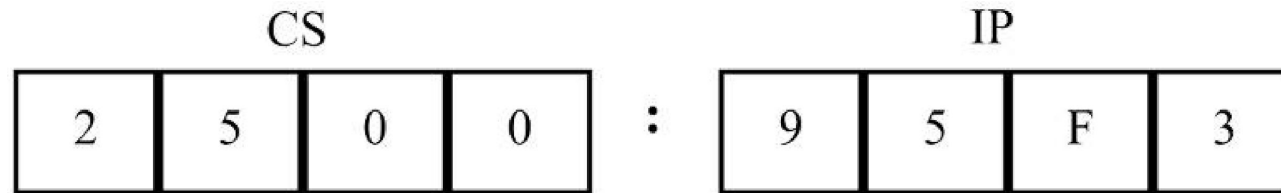
Para ejecutar un programa, 8086 obtiene las instrucciones (opcodes y operandos) del segmento de código.

- **La dirección lógica** de una instrucción siempre consiste en un CS (segmento de código) y una IP (puntero de instrucción), que se muestra en formato **CS : IP**.
- La dirección física para la ubicación de la instrucción se genera **desplazando el CS un dígito hexadecimal a la izquierda** y luego sumando a la IP. IP contiene la **dirección de desplazamiento**.

La dirección de 20 bits resultante se denomina **dirección física**, ya que esta se coloca en los pines del bus de direcciones físicas externas.

Segmento de código

- Suponga valores en CS e IP como se muestra en el diagrama



- La dirección offset contenida en IP, es 95F3H.
- La dirección lógica es CS:IP, o 2500:95F3H
- La dirección física será $25000 + 95F3 = 2E5F3H$

Ejemplo

Example 1-1

If $CS = 24F6H$ and $IP = 634AH$, show (a) the logical address, and (b) the offset address. Calculate (c) the physical address, (d) the lower range, and (e) the upper range of the code segment.

Ejemplo

Example 1-1

If CS = 24F6H and IP = 634AH, show (a) the logical address, and (b) the offset address. Calculate (c) the physical address, (d) the lower range, and (e) the upper range of the code segment.

Solution:

- | | | |
|--------------------------|--------------------------|--------------------------|
| (a) 24F6:634A | (b) 634A | (c) 2B2AA (24F60 + 634A) |
| (d) 24F60 (24F60 + 0000) | (e) 34F5F (24F60 + FFFF) | |

Dirección lógica/física del segmento de código

LOGICAL ADDRESS	MACHINE LANGUAGE	ASSEMBLY LANGUAGE
<u>CS:IP</u>	<u>OPCODE AND OPERAND</u>	<u>MNEMONICS AND OPERAND</u>
1132:0100	B057	MOV AL, 57
1132:0102	B686	MOV DH, 86
1132:0104	B272	MOV DL, 72
1132:0106	89D1	MOV CX, DX
1132:0108	88C7	MOV BH, AL
1132:010A	B39F	MOV BL, 9F
1132:010C	B420	MOV AH, 20
1132:010E	01D0	ADD AX, DX
1132:0110	01D9	ADD CX, BX
1132:0112	05351F	ADD AX, 1F35

La Instrucción "**MOV AL, 57**" tiene un código máquina de **B057**.
B0 es el opcode y 57 es el operando.

Dirección lógica vs dirección física en CS

CS:IP	Machine Language	Mnemonics
1132:0100	B057	MOV AL,57h
1132:0102	B686	MOV DH,86h
1132:0104	B272	MOV DL,72h
1132:0106	89D1	MOV CX,DX
1132:0108	88C7	MOV BH,AL
1132:010A	B39F	MOV BL,9F
1132:010C	B420	MOV AH,20h
1132:010E	01D0	ADD AX,DX
1132:0110	01D9	ADD CX,BX
1132:0112	05351F	ADD AX, 1F35h

<u>LOGICAL ADDRESS</u>	<u>PHYSICAL ADDRESS</u>	<u>MACHINE CODE CONTENTS</u>
1132:0100	11420	B0
1132:0101	11421	57
1132:0102	11422	B6
1132:0103	11423	86
1132:0104	11424	B2
1132:0105	11425	72
1132:0106	11426	89
1132:0107	11427	D1
1132:0108	11428	88
1132:0109	11429	C7

Segmento de datos

- Supongamos que el desplazamiento del segmento de datos comienza en 200H.
– Los datos se colocan en ubicaciones de memoria:

```
DS:0200 = 25
DS:0201 = 12
DS:0202 = 15
DS:0203 = 1F
DS:0204 = 2B
```

- El programa puede ser escrito así:

```
MOV  AL,0           ;clear AL
ADD  AL,[ 0200]      ;add the contents of DS:200 to AL
ADD  AL,[ 0201]      ;add the contents of DS:201 to AL
ADD  AL,[ 0202]      ;add the contents of DS:202 to AL
ADD  AL,[ 0203]      ;add the contents of DS:203 to AL
ADD  AL,[ 0204]      ;add the contents of DS:204 to AL
```

Segmento de datos

- La dirección de desplazamiento está entre paréntesis, lo que indica que el operando representa la dirección de los datos y no los datos en sí.

```
MOV  AL,0          ;clear AL
ADD  AL,[0200]      ;add the contents of DS:200 to AL
```

- Si no se incluyeran los paréntesis, "MOV AL,0200", la CPU intentaría mover 200 a AL en lugar del contenido de la dirección offset 200

Segmento de datos

- Si los datos tuvieran que almacenarse en una dirección offset diferente, el programa tendría que ser reescrito
 - Una forma de resolver este problema es usar un registro para mantener la dirección de desplazamiento, y antes de cada ADD, incrementar el registro para acceder al siguiente byte.
- El 8088/86 sólo permite el uso de registros BX, SI y DI como registros de desplazamiento para el segmento de datos
 - El término **puntero** se utiliza a menudo para un registro que contiene una dirección offset.

- En el siguiente ejemplo BX se usa como puntero:

```
MOV  AL,0           ;initialize AL
MOV  BX,0200H       ;BX points to offset addr of first byte
ADD  AL,[BX]        ;add the first byte to AL
INC  BX             ;increment BX to point to the next byte
ADD  AL,[BX]        ;add the next byte to AL
INC  BX             ;increment the pointer
ADD  AL,[BX]        ;add the next byte to AL
INC  BX             ;increment the pointer
ADD  AL,[BX]        ;add the last byte to AL
```

Example 1-2

Assume that DS is 5000 and the offset is 1950. Calculate the physical address.

Solution:

DS				:	offset			
5	0	0	0	:	1	9	5	0

The physical address will be $50000 + 1950 = 51950$.

1. Start with DS.

5	0	0	0
---	---	---	---

2. Shift DS left.

5	0	0	0	0
---	---	---	---	---

3. Add the offset.

+	1	9	5	0
---	---	---	---	---

4. Physical address.

5	1	9	5	0
---	---	---	---	---

Example 1-3

If DS = 7FA2H and the offset is 438EH, calculate (a) the physical address, (b) the lower range, and (c) the upper range of the data segment. Show (d) the logical address.

Solution:

(a) 83DAE ($7FA20 + 438E$)

(b) 7FA20 ($7FA20 + 0000$)

(c) 8FA1F ($7FA20 + FFFF$)

(d) 7FA2:438E

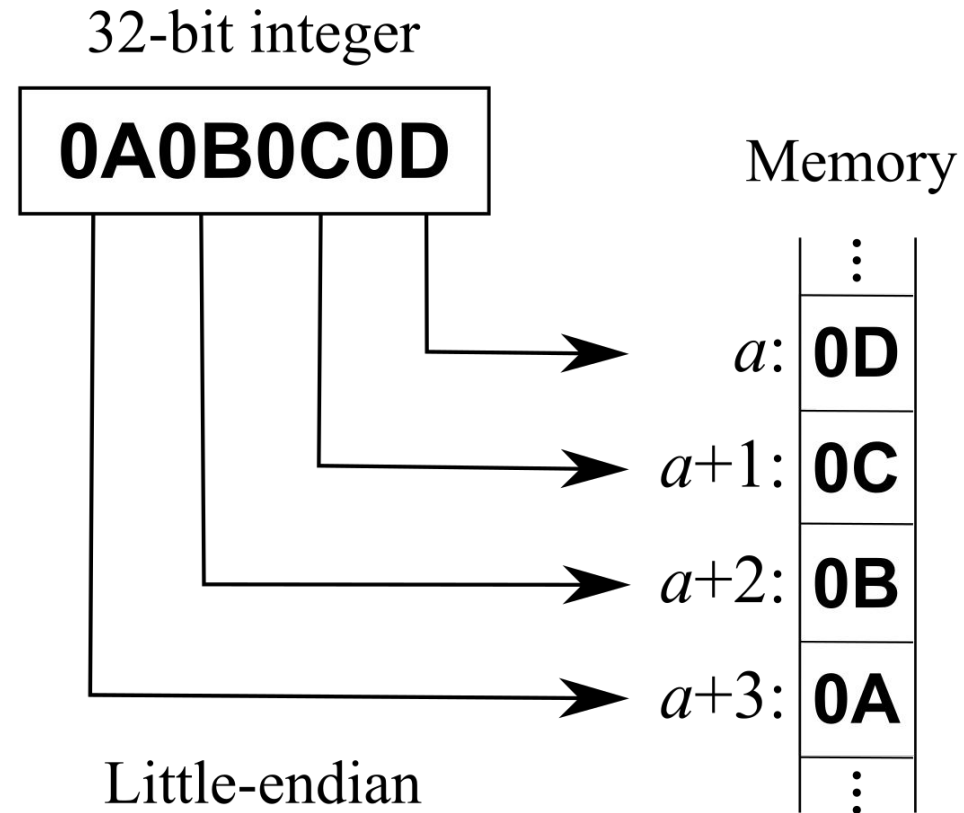
Example 1-4

Assume that the DS register is 578C. To access a given byte of data at physical memory location 67F66, does the data segment cover the range where the data resides? If not, what changes need to be made?

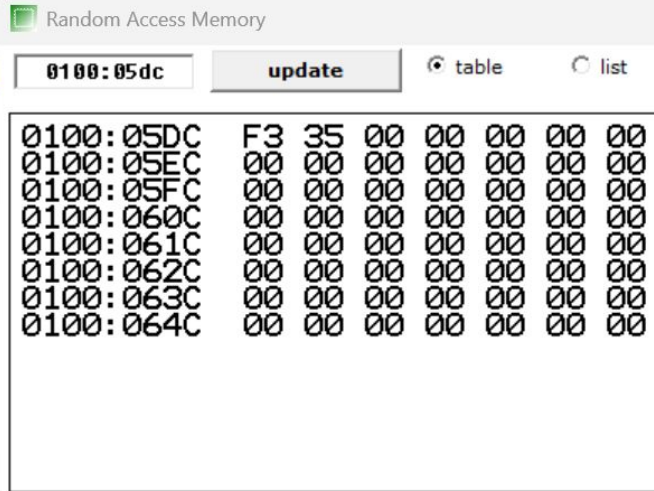
Solution:

No, since the range is 578C0 to 678BF, location 67F66 is not included in this range. To access that byte, DS must be changed so that its range will include that byte.

Convención little endian



- ```
MOV [1500] ,AX ;copy the contents of AX to offset 1500H
```



# Mapa de memoria

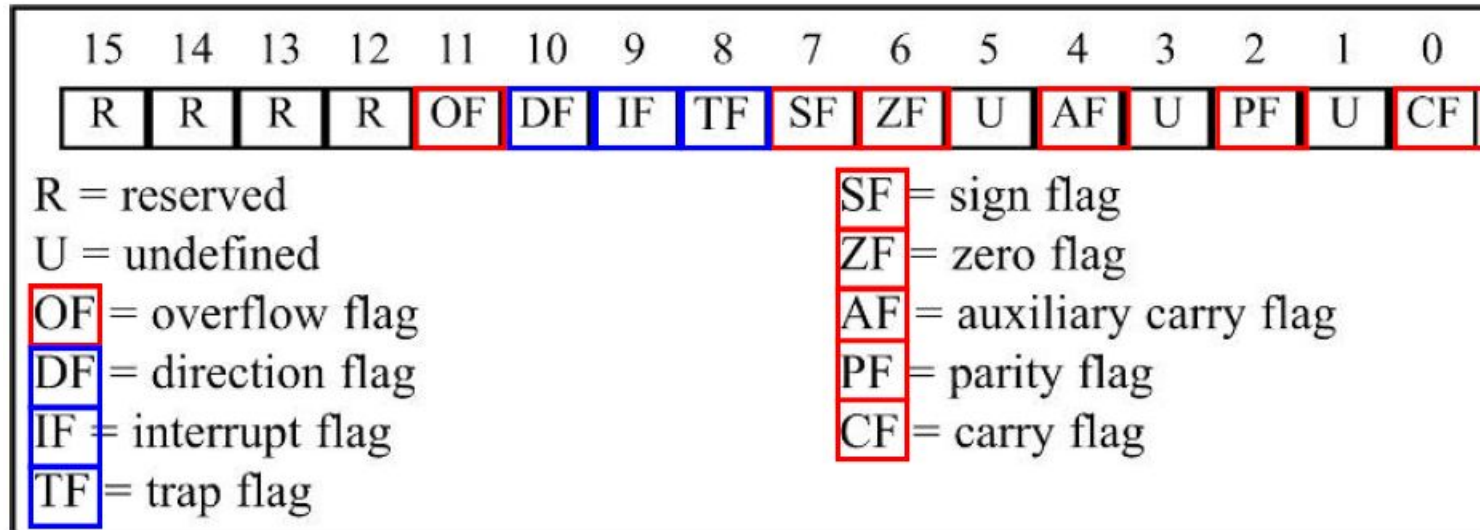
- La dirección de 20 bits de 8088/86 permite 1 MB (1024 KB bytes) de espacio de memoria con el intervalo de direcciones 00000–FFFFFF.
- Esta asignación de memoria se denomina **mapa de memoria**

|                           |         |
|---------------------------|---------|
| RAM<br>640K               | 00000H  |
|                           | 9FFFFH  |
| Video Display<br>RAM 128K | A0000H  |
|                           | BFFFFH  |
| ROM<br>256K               | C0000H  |
|                           | FFFFFFH |



# REGISTRO DE BANDERAS

- El registro de banderas es un registro de 16 bits, a veces denominado registro de estado.



# REGISTRO DE BANDERAS

## Uso del ZF para bucles

- Una aplicación ampliamente utilizada del registro de bandera es el uso de la bandera cero para implementar bucles de programa.
  - Un bucle es un conjunto de instrucciones repetidas varias veces.
- Como ejemplo, para SUMAR 5 bytes de datos, se puede usar un contador para realizar un seguimiento de cuántas veces se debe repetir el bucle.
- Cuando el contador se convierte en cero, se establece el indicador cero ( $ZF = 1$ ) y se detiene el bucle.

```
MOV CX,05 ;CX holds the loop count
MOV BX,0200H ;BX holds the offset data address
MOV AL,00 ;initialize AL
ADD_LP: ADD AL,[BX] ;add the next byte to AL
 INC BX ;increment the data pointer
 DEC CX ;decrement the loop counter
 JNZ ADD_LP ;jump to next iteration if counter
 not zero
```

# REGISTRO DE BANDERAS

## Uso del ZF para bucles

- El Registro **CX** se utiliza para sostener el contador.
  - **BX** es el puntero de desplazamiento.
  - (SI o DI podrían haberse utilizado en su lugar)



```

ADD_LP: MOV CX,05 ;CX holds the loop count
 MOV BX,0200H ;BX holds the offset data address
 MOV AL,00 ;initialize AL
 ADD AL,[BX] ;add the next byte to AL
 INC BX ;increment the data pointer
 DEC CX ;decrement the loop counter
 JNZ ADD_LP ;jump to next iteration if counter
 not zero

```

# REGISTRO DE BANDERAS

## Uso del ZF para bucles

- El Registro **CX** se utiliza para sostener el contador.
  - **BX** es el puntero de desplazamiento.
- (SI o DI podrían haberse utilizado en su lugar)

```
MOV CX,05 ;CX holds the loop count
MOV BX,0200H ;BX holds the offset data address
MOV AL,00 ;initialize AL
ADD_LP: ADD AL,[BX] ;add the next byte to AL
 INC BX ;increment the data pointer
 DEC CX ;decrement the loop counter
 JNZ ADD_LP ;jump to next iteration if counter
 not zero
```

# MODOS DE DIRECCIONAMIENTO x86

- La CPU puede acceder a los operandos (datos) de varias maneras, llamadas modos de direccionamiento.
  - El número de modos de direccionamiento se determina cuando se diseña el microprocesador y no se puede cambiar
- The x86 provides seven distinct addressing modes:
  - – 1 - Register
  - – 2 - Immediate
  - – 3 - Direct
  - – 4 - Register indirect
  - – 5 - Based relative
  - – 6 - Indexed relative
  - – 7 - Based indexed relative

# MODOS DE DIRECCIONAMIENTO x86

**Table 1-7: Summary of the x86 Addressing Modes**

| Addressing Mode        | Operand       | Default Segment |
|------------------------|---------------|-----------------|
| Register               | reg           | none            |
| Immediate              | data          | none            |
| Direct                 | [offset]      | DS              |
| Register indirect      | [BX]          | DS              |
|                        | [SI]          | DS              |
|                        | [DI]          | DS              |
| Based relative         | [BX]+disp     | DS              |
|                        | [BP]+disp     | SS              |
| Indexed relative       | [DI]+disp     | DS              |
|                        | [SI]+disp     | SS              |
| Based indexed relative | [BX][SI]+disp | DS              |
|                        | [BX][DI]+disp | DS              |
|                        | [BP][SI]+disp | SS              |
|                        | [BP][DI]+disp | SS              |

# La PILA (Stack)

La pila es una sección de memoria de lectura/escritura (RAM) utilizada por la CPU para almacenar información temporalmente.

- La CPU necesita esta área de almacenamiento ya que solo hay un número limitado de registros.

Debe haber algún lugar para que la CPU almacene información de forma segura y temporal.

La principal desventaja de la pila es el **tiempo de acceso**.

- Dado que la pila está en RAM, tarda mucho más en acceder en comparación con el tiempo de acceso a los registros.

Los registros que apuntan a la pila son:

- El registro SS (stack segment).

- El registro SP (stack pointer).

Estos registros deben cargarse antes de utilizar cualquier instrucción para acceder a la pila.

- El almacenamiento de un registro de la CPU en la pila se denomina **push**.

- La carga del contenido de la pila en un registro de la CPU se denomina **pop**.

# La PILA (Stack)

**El registro de puntero de pila (SP) x86 apunta a la ubicación de memoria actual utilizada como parte superior de la pila.**

- A medida que los datos se introducen en la pila, esta se decrementa.**
- A medida que los datos se extraen de la pila a la CPU, esta se incrementa.**

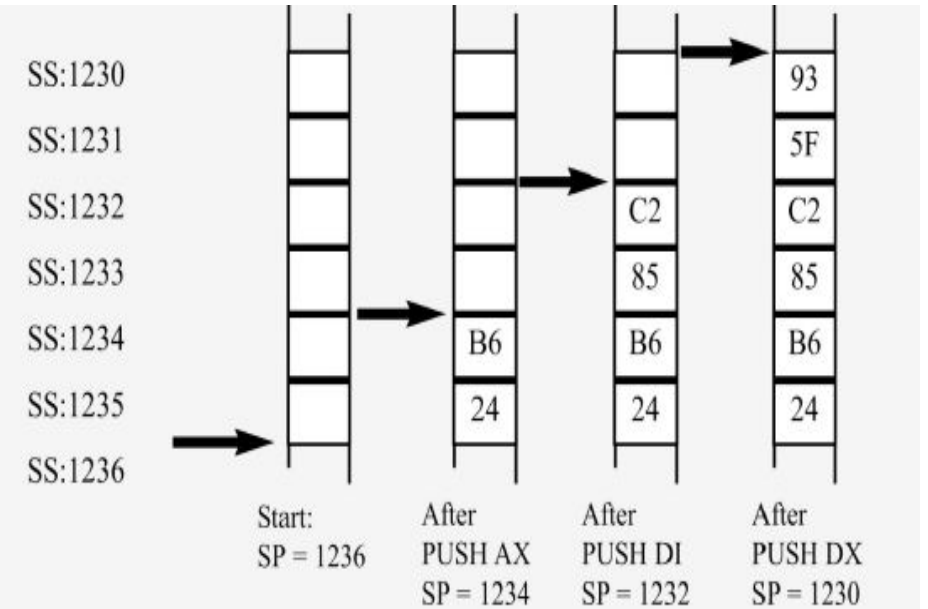
**El “SP” es decrementado luego de un PUSH, para asegurarse que la pila crezca hacia abajo es decir, de direcciones altas a direcciones mas bajas.**

**Es decir, es lo contrario al Registro IP. (Instruction pointer)**

# PUSH en la pila

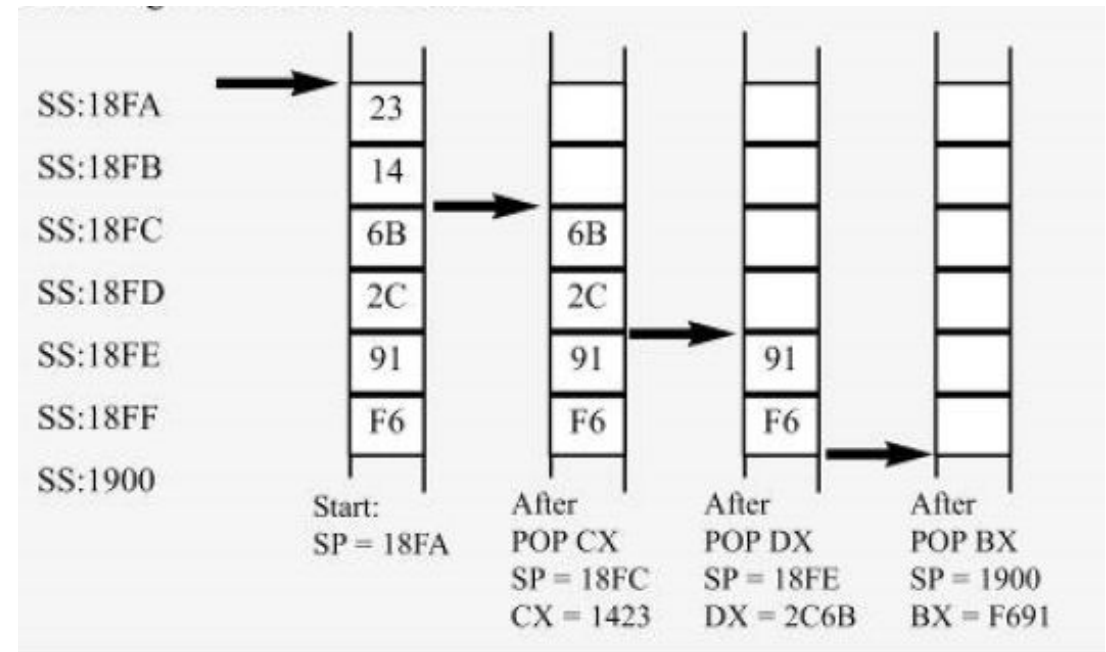
- A medida que se ejecuta cada PUSH, el contenido del registro se guarda en la pila y el SP se decrementa el "SP" en 2.
- Asumiendo que SP=1236, AX=24B6, DI=85C2, y DX=5F93, muestre los contenidos de las siguientes instrucciones ejecutadas:
- PUSH AX
- PUSH DI
- PUSH DX

Solution:



# POP en la pila

- With every pop, the top 2 bytes of the stack are copied to the x86 CPU register specified by the instruction & the stack pointer is incremented twice
- Asumiendo que SP=18FA, muestre los contenidos de la pila y los registros para siguientes instrucciones ejecutadas:
- POP CX
- POP DX
- POP BX



# REGISTROS DEL 8088/8086

# DENTRO DEL 8088/86. REGISTROS

En la CPU, los registros almacenan información temporalmente.

- Uno o dos bytes de datos para procesar.

Se puede acceder a los registros de propósito general en procesadores 8088/86 como registros de 16 bits u 8 bits

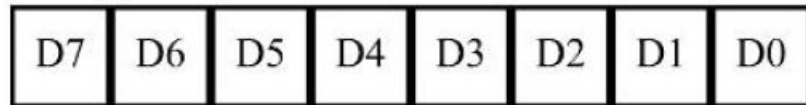
- Se puede acceder a todos los demás registros solo como los **16 bits completos**.

En 8088/86, los tipos de datos son de 8 o 16 bits

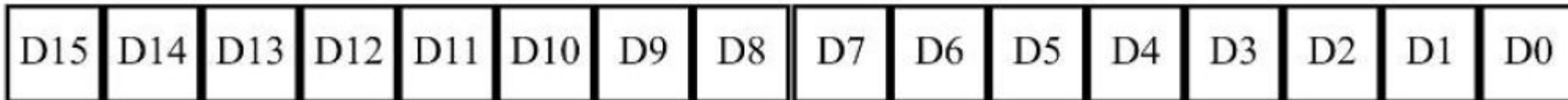
# DENTRO DEL 8088/86. REGISTROS

- Los bits de un registro están numerados en orden descendente, como se muestra:

8-bit register:

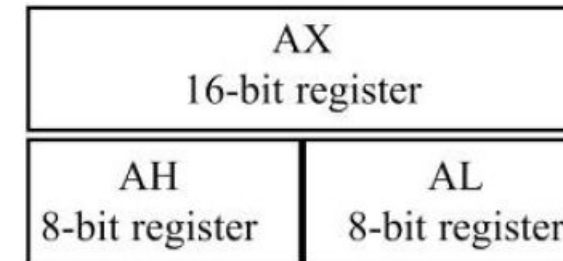


16-bit register:



# DENTRO DEL 8088/86. REGISTROS

- La primera letra de cada registro indica su uso.
- AX se utiliza para el acumulador  
BX es un registro de direccionamiento base  
CX es un contador en operaciones de bucle  
DX apunta a datos en operaciones de E/S



# DENTRO DEL 8088/86. REGISTROS POR CATEGORÍA

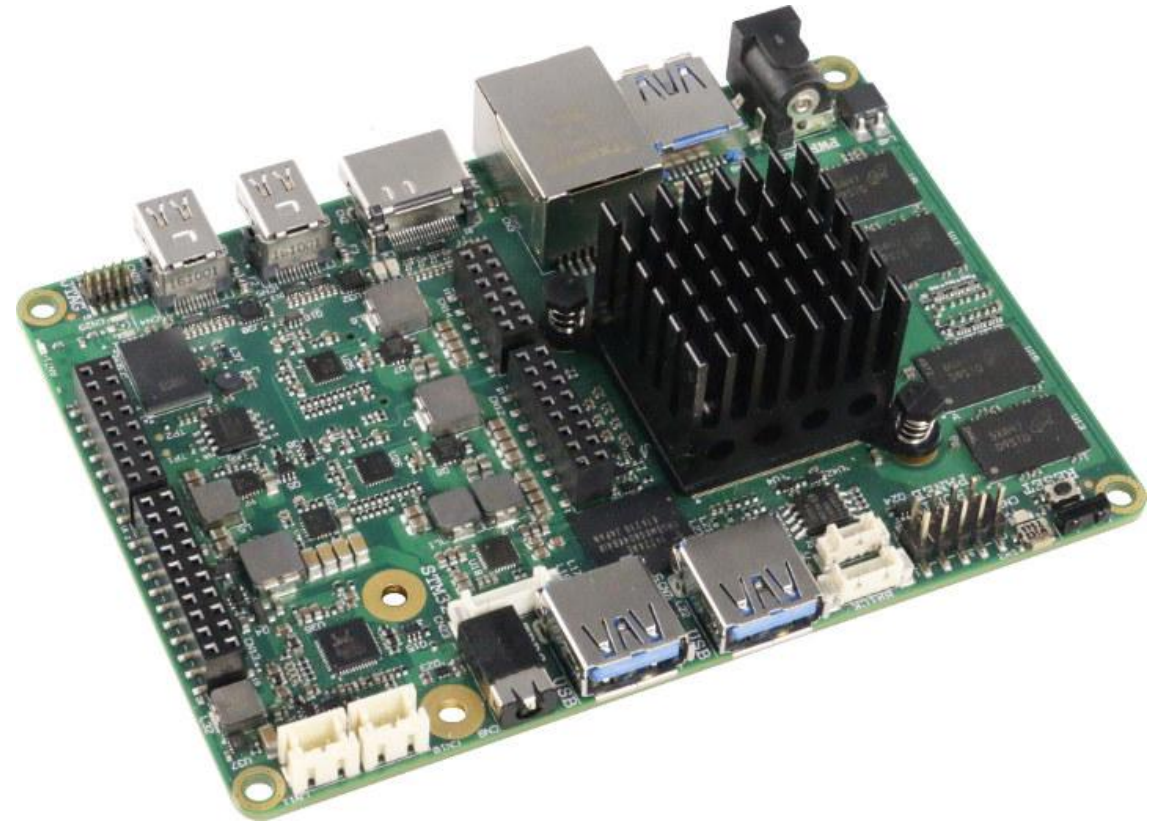
| CATEGORÍA   | Bits | NOMBREDELREGISTRO                       |
|-------------|------|-----------------------------------------|
| CATEGORÍA   | Bits | NOMBREDELREGISTRO                       |
| General     | 16   | AX,BX,CX,DX                             |
| General     | 8    | AH,AL,BH,BL,CH,CL,DH,DL                 |
| Pointer     | 16   | SP (Stack Pointer),Base Pointer(BP)     |
| Index       | 16   | SI(Source Index),DI (Destination Index) |
| Segment     | 16   | CS(Code Segment)                        |
| Segment     | 16   | DS(Data Segment)                        |
| Segment     | 16   | SS(Stack Segment)                       |
| Segment     | 16   | ES(Extra Segment)                       |
| Instruction | 16   | IP(Instruction Pointer)                 |
| Flag        | 16   | FR(Flag Register)                       |

# Registros de puntero e índice

|           |                            |
|-----------|----------------------------|
| <b>SP</b> | <b>Stack Pointer</b>       |
| <b>BP</b> | <b>Base Pointer</b>        |
| <b>SI</b> | <b>Source Index</b>        |
| <b>DI</b> | <b>Destination Index</b>   |
| <b>IP</b> | <b>Instruction Pointer</b> |

# Resumen de registros

| CATEGORÍA   | Bits | NOMBRE DEL REGISTRO                     |
|-------------|------|-----------------------------------------|
| General     | 16   | AX,BX,CX,DX                             |
| General     | 8    | AH,AL,BH,BL,CH,CL,DH,DL                 |
| Pointer     | 16   | SP (Stack Pointer),Base Pointer(BP)     |
| Index       | 16   | SI(Source Index),DI (Destination Index) |
| Segmento    | 16   | CS(Code Segment)                        |
| Segmento    | 16   | DS(Data Segment)                        |
| Segmento    | 16   | SS(Stack Segment)                       |
| Segmento    | 16   | ES(Extra Segment)                       |
| Instruction | 16   | IP(Instruction Pointer)                 |
| Flag        | 16   | FR(Flag Register)                       |



Esta foto de Autor desconocido está bajo licencia [CC BY-SA](#)

**GRACIAS**

# Práctica

- Realice una serie de instrucciones que sume 4 números de 16 bits y lo guarde en una variable suma.
  - Datos: 234DH, 1DE6H, 3BC7H, 566AH
  - guardar datos en otro registro.
- Identifique la dirección física de cada Instrucción y la dirección lógica.